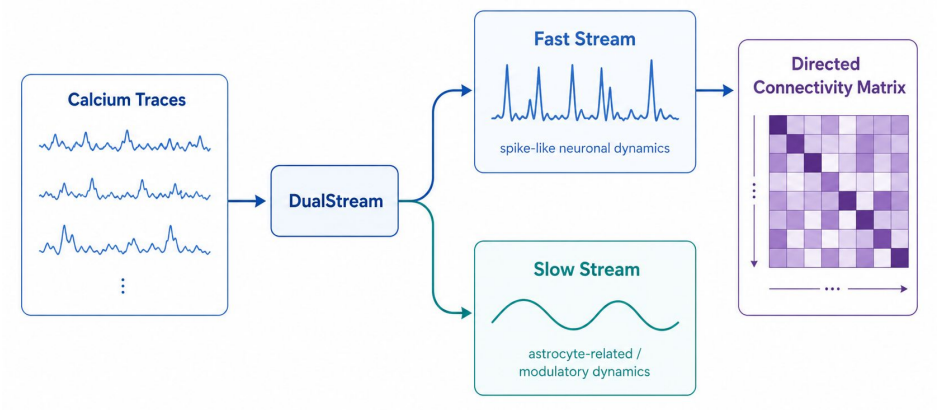
Losert Lab

UNIVERSITY OF MARYLAND

Dual Stream

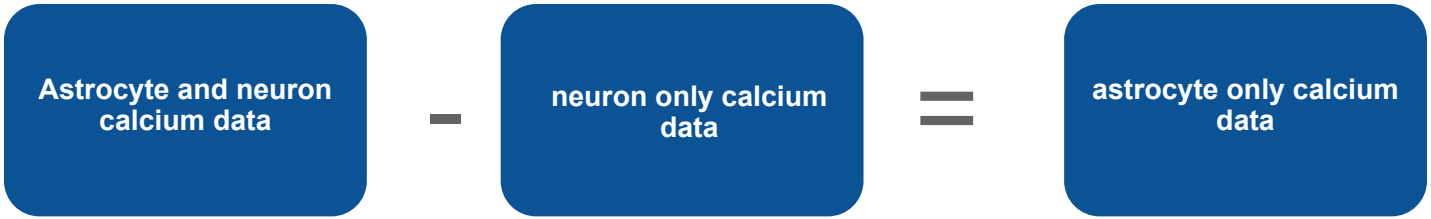
End-to-End Functional Connectivity Inference from Calcium Imaging via Dual-Timescale Selective State Space Models



Aneesh Singh and Shohini Sarkar

Motivation

- After collecting calcium fluorescence data, we used FIJI and Python for further downstream analysis
- There was subsequently difficulty in analyzing the calcium fluorescence data that was generated over the summer
  - Requirement to use “subtraction” to get the true astrocyte data to analyze
  - Afterwards use spike detection libraries (Cascade, Template matching) to analyze
  - Not guaranteed to have the most accurate dataset of calcium fluorescence
  - Brain data is incredibly noisy



Motivation

- neurons represent the fast nodes in the neural network; analyzing its activity gives us the ability to understand how information is encoded and transmitted to the brain
- Astrocytes (the slow nodes) have the ability to modulate the neuronal activity
- Traditionally - we convert the fluorescence to spikes and then use existing libraries for peak detection
    - Granger causality used to determine how different neurons influence each other
    - Peak detection methods used to show that a spike has occurred or to match a peak
- With this “newer” theoretical pipeline, we would go from a raw video to an immediate connectivity matrix
    - Would ideally avoid errors by skipping errors
    - Involves a dual time-scale model that treats the brain as an integrated system

Previous Pipeline

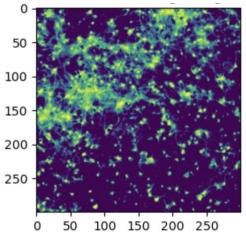
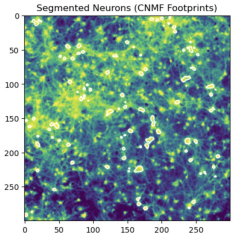
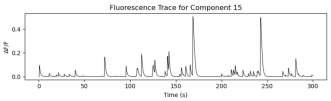


Image Cells

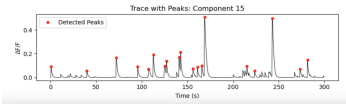


Segmentation

Analyzing a cell's fluorescence helps us understand its activity!



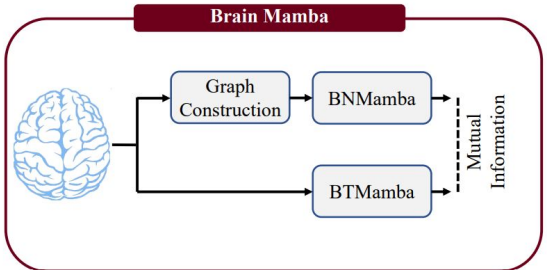
Extract Fluorescence



Analyze Fluorescence Traces

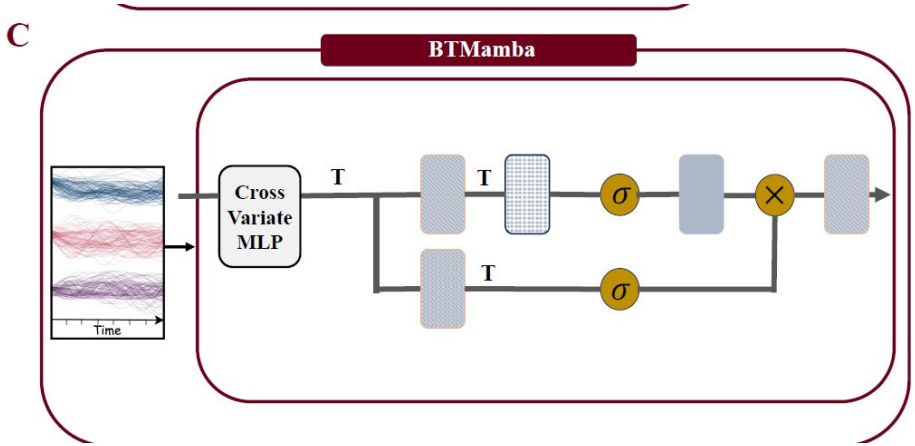
Brain MAMBA

- Deep learning model that is used for integrating spatial and temporal data information
- According to studies, it was primarily used for fMRI and EEG studies
    - Existing models pose the problems of having high computational costs of Transformers and inability to capture long-range dependencies
    - Data falls victim to oversquashing which entails the data being put into small, simplified vectors
    - Too much data = Out of Memory" (OOM) error
- Uses a selective state model and operates on a linear scalability model (rather than quadratic)
- Features two components: BTMamba and BNMamba
- More scalable and memory efficient; filters out noise and focuses only at informative timestamps



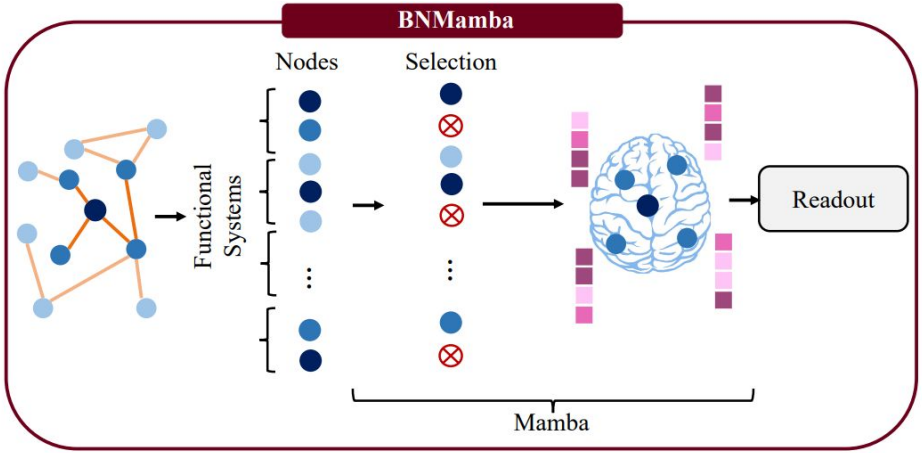
Brain Timeseries Mamba (BTMamba)

- Serves as the temporal encoder branch
- Looks at the different time and brain units (variates) and uses a Multilayer Perceptron to capture and fuse the variates
- After fusing, it then uses an input dependent state space model to encode each variate and pay attention to informative timestamps
- Operates with linear time and memory complexity
- Other models tend to treat the variate information separately and miss out on examining how activity in one region can influence the other
- Allows for a cohesive understanding of the “full circuit”



Brain Network MAMBA (BNMamba)

- Uses a a novel graph learning framework that uses message-passing neural network (MPNN) to first learn the local dependencies of brain units (e.g., voxels or channels)
    - Gives a better understanding of how immediate neighbors in the brain circuit interact
- then uses a selective space state model to aggregate the information across the entire network
    - Prevents oversquashing
- Treats the brain as an organized series of graph tokens



Capabilities

- achieves superior performance with respect to the state-of the art methods
- each of the encoders ALONE also outperforms their corresponding baselines
- achieved exceptional performance in anomaly detection
- Uses a bidirectional readout to show that the order in which the cells are processed does not bias the final connectivity
- Shows that the selective space state model is more effective than other traditional graphs or time series encoders
- Also suggests that the architecture is more biologically correct and relies less on raw computing power to find patterns

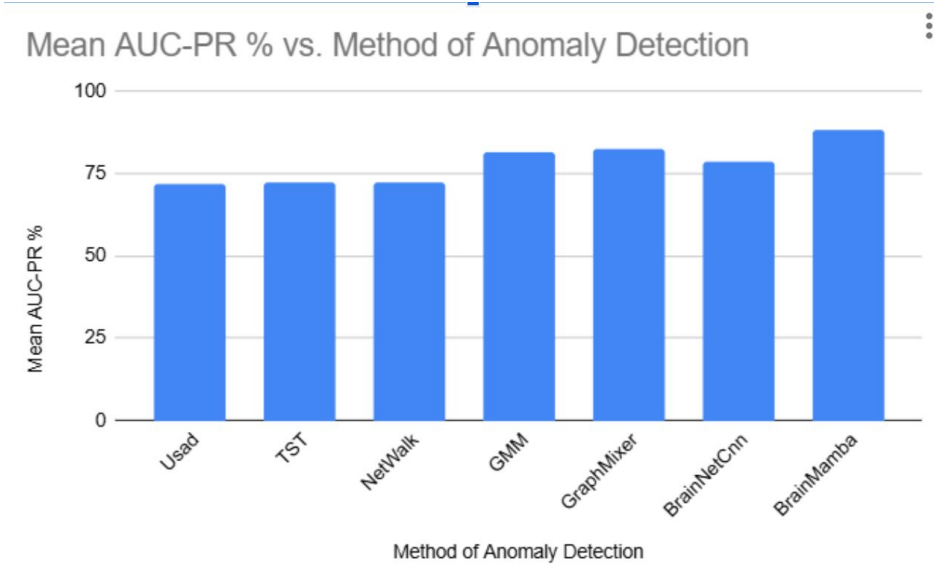


Table 1: The comparison of recent brain activity encoders.

	MVTS (Potter et al., 2022)	BNTRANSFORMER (Kan et al., 2022b)	GRAPHS4MER (Tang et al., 2023)	PTGB (Yang et al., 2023)	BRAINMIXER (Behrouz et al., 2023)	BRAINMAMBA (This work)
Data	EEG (Timeseries)	fMRI (Graph)	EEG† (Timeseries + Graph)	fMRI (Graph)	All‡ (Timeseries + Graph)	All‡ (Timeseries + Graph)
Brain Unit	Channels	ROIs	Channels	ROIs	Highly active voxels (Channels in EEG)	All voxels (Channels in EEG)
Spatial Encoding	-	✓	✓	✓	✓	✓
Temporal Encoding	✓	-	✓	-	✓	✓
Inter-variate Information Fusing	-	-	-	-	✓	✓
Pre-training	✓	-	-	✓	✓	✓
Long-range Sequence	-	-	✓	-	-	✓
Time Complexity	Quadratic	Quadratic	Quadratic	Quadratic	Quadratic	**Linear**

† Although GRAPHS4MER can potentially be employed on neuroimage modalities with low sampling frequency (e.g. fMRI), it is best suited for timeseries with long-range temporal dependencies (e.g., EEG) as discussed by Tang et al. (2023).

‡ All fMRI, EEG, MEG, and generally any neuroimaging modalities that provide multivariate timeseries recorded from multiple units across the brain.

How can we apply this to our research?

- Use the same “architecture” of BRAINMAMBA and apply it to the astrocyte-neuronal stimulation projects
  - Utilize the dual time-scale model that can examine BOTH the neuronal and astrocyte data
  - Do not need to undergo bleach correction or deconvolution
- Use aspects of BNMamba to show how each cell is a node and portray the spatial relationships between the cells
  - use a Pearson correlation to see whether “cell A influences cell B” (do they glow at similar times)
- Use aspects of BTMamba to “fuse” the astrocyte and neuronal signals together and see how the slow astrocytic modulation relates to the fast neuronal spiking
  - Examine whether after light stimulation, the astrocytes appeared as high order nodes

Pipeline Overview

1.  start with DF/f calcium traces
2.  preprocess the traces
3.  run baseline methods
4.  encode temporal patterns
5.  split into fast and slow streams
6.  compute loss & update
7.  compute connectivity
8.  return final matrix!

Data Preprocessing

Input: ΔF/F calcium traces shaped [neurons, time]

Preprocessing steps:
1. Load traces from .npy, .csv, or .hdf5 files
2. Standardize orientation to [neurons, time]
3. Handle missing values by interpolating NaNs
4. Normalize each neuron independently
5. Split long recordings into overlapping temporal windows
6. Store metadata like session ID, frame rate, and condition

Final model input:
[batch, neurons, time]

```
X = load_trace_file(file_path)
X = standardize_trace_shape(X,
expected_orientation="auto")
X = handle_missing_values(X)
X = normalize_per_neuron(X)

windows =
segment_into_windows(
    X,
    window_size=500,
    stride=250,
)
```

Baseline Methods

Pearson Correlation

Measures how similarly two neurons' calcium traces rise and fall together, but it is undirected and can be confused by shared slow signals.

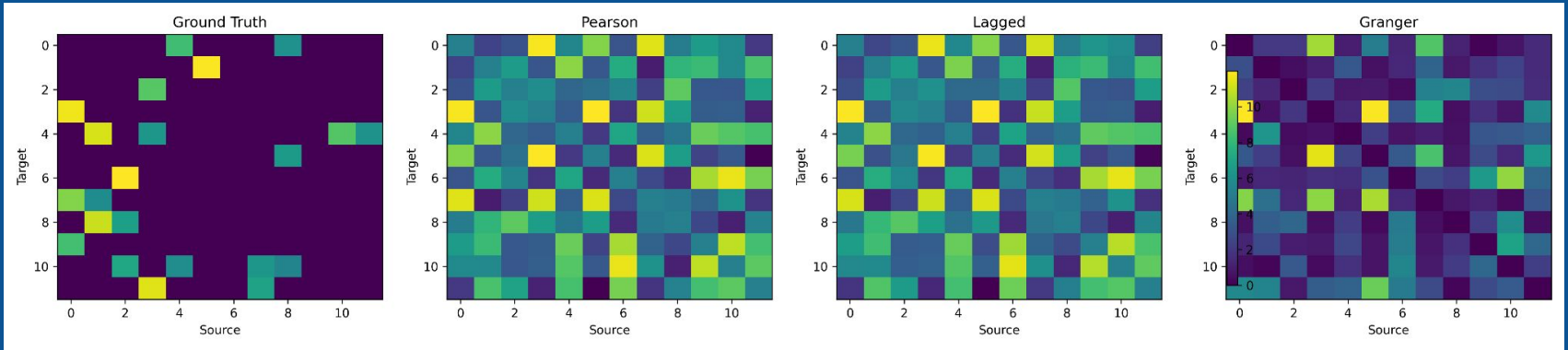
Lagged Correlation

Compares one neuron's past activity to another neuron's current activity.

Grainger Casuality

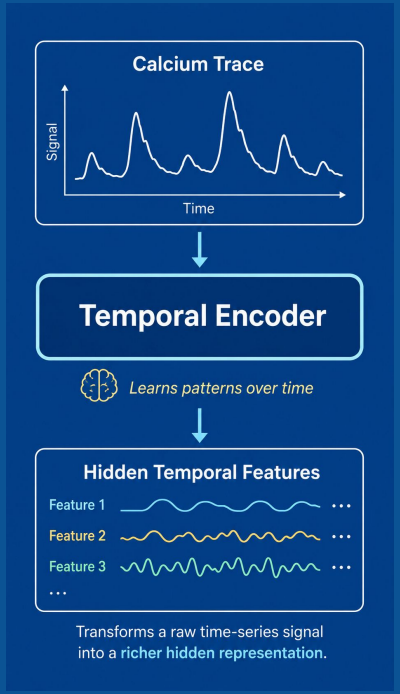
Tests whether including the past activity of a source neuron improves prediction of a target neuron's future activity.

Baseline Methods



Temporal Encoder

- The temporal encoder looks at each neuron's calcium trace as a sequence over time, not just as isolated points
- Its job is to learn patterns in the trace, such as rises, falls, bursts, slow changes, and repeated activity patterns
- In this prototype, we use a GRU, which is a simple neural network module that keeps a running "memory" of what happened earlier in the trace



Temporal Encoder

- This matters because neural activity is time-dependent: what happens now often depends on what happened a few moments before
- After the temporal encoder, each neuron's trace is represented in a richer hidden form that the model can then split into fast and slow components

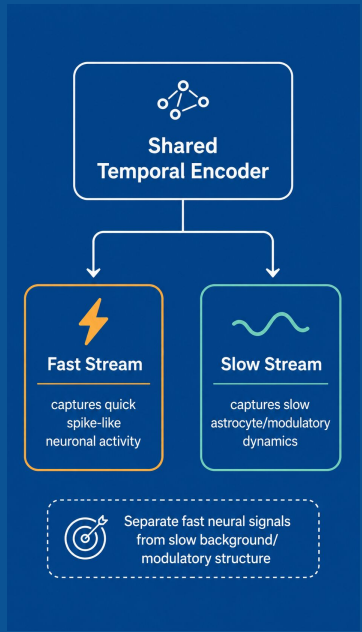
```
# Treat each neuron as its own time-series sequence,
# while sharing the same encoder weights across neurons.
x_flat = x.reshape(batch_size * num_neurons, timepoints, 1)


h = self.input_projection(x_flat)

# Learn temporal patterns using the shared encoder.
shared, _ = self.shared_encoder(h)
```

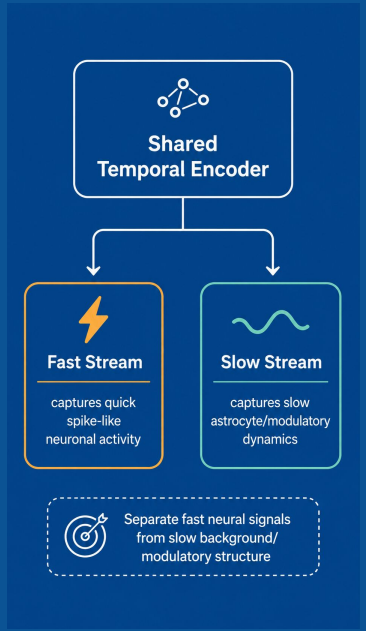
Fast & Slow Streams

- After the shared temporal encoder, the model splits the calcium trace representation into two separate streams.
- The **fast stream** is designed to capture quick, spike-like neuronal activity.
- The **slow stream** is designed to capture slower astrocyte-related, modulatory, or background dynamics.
- This split is useful because calcium traces are mixed signals, not pure neuronal spike recordings.



Fast & Slow Streams

- The fast stream is used to estimate directed neuron-to-neuron connectivity.
- The slow stream helps absorb slower structure that might otherwise confuse the connectivity estimate.
- In the current prototype, both streams are implemented with GRUs as lightweight stand-ins for future Mamba/selective state-space modules.
- The goal is to separate “what looks like fast neural communication” from “what looks like slower shared modulation.”



Loss & Updates

- Several losses are calculated and combined into a weighted loss

  - Backpropagation computes how to change the model weights, and the optimizer updates them to reduce the total loss.
  - During training, validation loss is tracked, gradients are clipped for stability, and the best model checkpoint is saved.

Compute Connectivity

- The goal is to build a directed connectivity matrix,
- The model computes connectivity from the **fast latent stream**, not directly from the raw calcium traces.
- For each neuron pair, it compares the **source neuron's past fast activity** to the **target neuron's current fast activity**.

Future Steps

- Want to update the architecture to use actual MAMBA
- Run the model on real data to get usable results
- Do much more training & tuning beyond the initialized parameters

References

Behrouz, Ali, and Farnoosh Hashemi. “Brain-Mamba: Encoding Brain Activity via Selective State Space Models.” *PMLR*, 24 July 2024, pp. 233–250, proceedings.mlr.press/v248/behrouz24a.html. Accessed 13 May 2026.